Transformers: 2017 vs 2026
(Multimodal LLMs, MoE, Attention at Scale)

Naeemullah Khan
naeemullah.khan@kaust.edu.sa

جامعة الملك عبدالله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

July 16, 2026

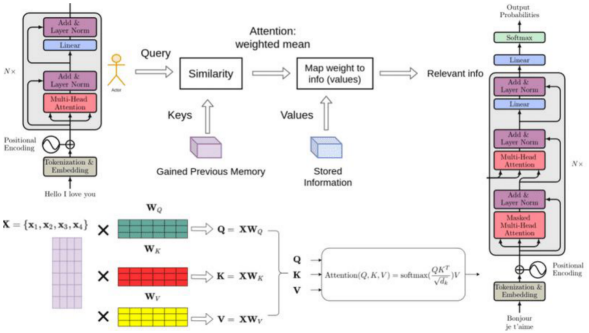
Table of Contents

Table of Contents (cont.)

Four Key Engineering Changes (2017 → 2026)

1. **Bandwidth matters more than just speed:** New attention methods like FlashAttention focus on moving data efficiently and using memory well.
2. **From fixed to flexible compute:** Models now adjust their computation and memory use based on the input (using tools like sparse MoE and adaptive resolution).
3. **From separate to unified input streams:** Instead of having different pieces for text, images, etc., everything is processed together in one sequence, with new practical challenges.
4. **More thinking during use:** Some new models use extra internal steps ("thinking tokens") for harder questions, shifting cost to longer decoding instead of just bigger models.

The 2017 Transformer



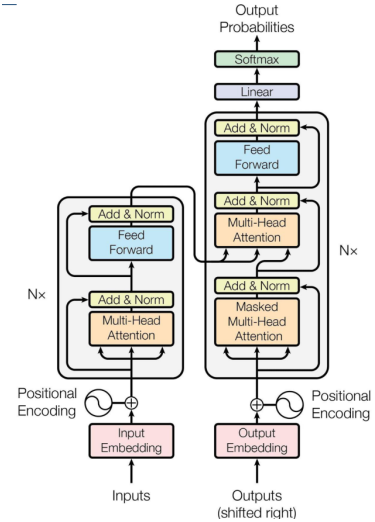
Why the Transformer?

- The original 2017 Transformer was designed to address key limitations of Recurrent Neural Networks (RNNs) and Long Short-Term Memory (LSTM) units:
  - Lack of parallelization
  - Vanishing gradient problem inherent in sequential processing
- Replaced recurrence with self-attention to allow all tokens in a sequence to be processed simultaneously during training[1]
- Enabled significant reductions in wall-clock time needed for model convergence

[1] See Vaswani et al., "Attention is All You Need," 2017.

The 2017 Building Blocks (Recap)

- Tokenization and input embeddings
- Sinusoidal position encodings
- Self-attention over Query, Key, Value
- Multi-head attention
- Feed-forward blocks with Add & Norm
- Encoder stack, decoder stack, masked and cross-attention



Scaled Dot-Product and Multi-Head Attention (Recap)

Scaled dot-product attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Multi-head attention:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)W^O, \quad \text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

- Every token attends to every other token: quadratic cost in sequence length.
- In 2017 every head carries its own full K and V projections; MQA and GQA later relax this.

2017: Sinusoidal Positional Encoding (Recap)

- Self-attention is permutation-invariant: without extra signal, the model cannot tell token order.
- The 2017 Transformer adds a fixed sinusoidal vector to each input embedding.

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{\frac{2i}{d_{\text{model}}}}}\right)$$
$$PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{\frac{2i}{d_{\text{model}}}}}\right)$$

- The encoding is absolute: each position gets its own vector, added once at the input.
- Two problems surfaced at scale: absolute positions generalize poorly beyond the training length, and the signal is injected only once at the input, so every deeper layer receives it indirectly.

2026: From Absolute Positions to RoPE and ALiBi

- **Learnable position embeddings** (BERT, GPT-2): replace the fixed sinusoids with trained vectors. Adapt to the task, but still absolute and still capped at the training length.
- **Rotary Position Embeddings (RoPE)** (Su et al., 2021): instead of adding a position vector at the input, rotate each query and key by a position-dependent angle inside every attention layer.
  - The dot product between a query at position m and a key at position n then depends only on the offset $m - n$: relative position, injected at every layer.
  - Rotary frequencies can be rescaled after training (position interpolation, YaRN) to stretch the context window.
  - RoPE is the dominant choice in 2026: LLaMA, Mistral, and Qwen all use it.
- **ALiBi** (Press et al., 2022): no position embedding at all. Add a linear penalty to attention scores proportional to token distance, so attention decays with distance by construction. Extrapolates to longer sequences than it was trained on; used by BLOOM and MPT.

The Transformer Block and LayerNorm Placement (Recap)

The 2017 block:

- Multi-head attention
- Add & Norm
- Feed-forward layer
- Add & Norm

Post-LN in the original paper: normalization applied after each sublayer.

Feature	Pre-LN	Post-LN
Stability	✓ More stable	✗ Less stable
Gradient flow	✓ Better	✗ Can explode/vanish
Used in	GPT-3, T5	Original Transformer

By 2020 nearly every large model had moved the norm before the sublayer (Pre-LN): gradients reach early layers directly through the residual path, so deep stacks train without warm-up tricks.

2026: From LayerNorm to RMSNorm

LayerNorm (Ba et al., 2016; used in the 2017 Transformer)**:** center, then scale.

$$\mathrm{LN}(x) = \frac{x - \mu}{\sigma} \odot \gamma + \beta$$

RMSNorm (Zhang & Sennrich, 2019)**:** drop the mean subtraction and the bias; scale by the root mean square alone.

$$\mathrm{RMSNorm}(x) = \frac{x}{\mathrm{RMS}(x)} \odot \gamma, \qquad \mathrm{RMS}(x) = \sqrt{\frac{1}{d}\sum_{i=1}^{d} x_i^2}$$

2026: From LayerNorm to RMSNorm (cont.)

- One reduction over the hidden dimension instead of two, and no bias parameters: cheaper at every layer of every token.
- Quality matches or slightly beats LayerNorm at scale, so centering adds cost without benefit.
- LLaMA, Mistral, and Qwen all combine Pre-LN placement with RMSNorm, the default in 2026.
- Newer stacks also normalize inside attention: Qwen3, Gemma 3, and OLMo 2 apply RMSNorm to queries and keys (QK-Norm) to keep attention logits stable.

2017: The ReLU Feed-Forward Block (Recap)

- Each Transformer block ends with a position-wise feed-forward network: two linear maps with a ReLU between them.

$$\mathrm{FFN}(x) = \max(0,\ xW_1 + b_1)\, W_2 + b_2$$

- The hidden width is typically $4 \times d_{\text{model}}$, so most of a Transformer's parameters sit in these blocks.
- Attention mixes information across tokens; the feed-forward network transforms each token on its own, with the same weights at every position.

2026: From ReLU to SwiGLU

Gated Linear Units (Shazeer, 2020)**:** replace the single hidden activation with the product of a value path and a gate path.

$$\mathrm{FFN}_{\mathrm{SwiGLU}}(x) = \big(\mathrm{SiLU}(xW_1) \odot xW_3\big)\, W_2, \qquad \mathrm{SiLU}(z) = z \cdot \sigma(z)$$

- Three weight matrices instead of two; the hidden width shrinks to roughly $\frac{2}{3}$ of $4 \times d_{\text{model}}$ so the parameter count stays matched.
- Small but consistent quality gains at equal parameters and compute. The paper offers no theory for why: "we attribute their success, as all else, to divine benevolence."
- Bias terms are dropped at the same time: modern stacks remove b from attention and feed-forward projections with no measurable quality loss.
- LLaMA, Mistral, Qwen, and PaLM all use SwiGLU; ReLU feed-forward blocks have essentially disappeared from frontier models.

Shazeer, "GLU Variants Improve Transformer," arXiv:2002.05202.

KV Cache Memory

- In decode, each new token needs the key and value tensors of all earlier positions; caching them avoids recomputation, one cache per layer.
- Serving memory consists of model weights plus the KV cache, and under batching every request needs its own cache, often the limiting factor before weights.
- Cost grows linearly with batch size and context length, capping throughput and long-context use; MQA and GQA target this directly.

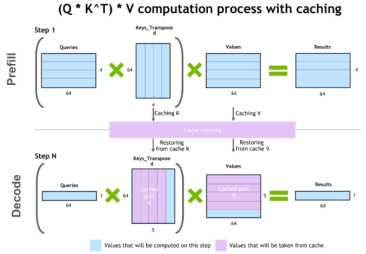
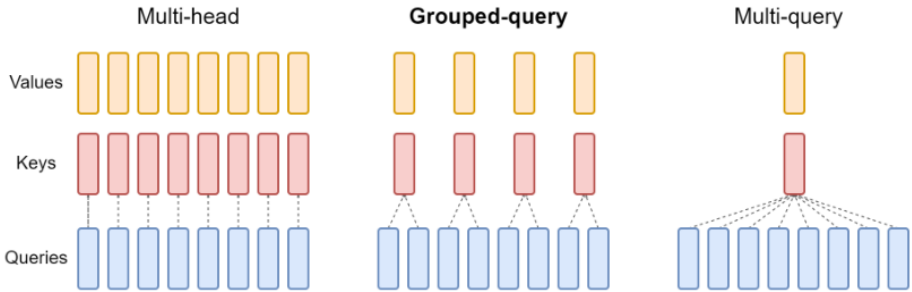


Illustration of the key-value caching mechanism. Verma & Vaidya, NVIDIA Technical Blog (2023).

MHA, MQA, GQA: shrinking the KV cache

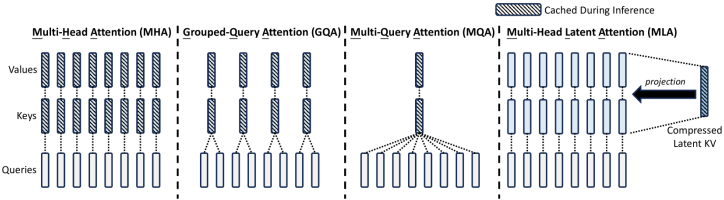


- **Multi-head attention (MHA)**: H separate query, key, and value heads.
- **Multi-query attention (MQA)**: single shared key and value heads for all query heads.
- **Grouped-query attention (GQA)**: shares key and value heads for each *group* of query heads.
- GQA conceptually interpolates between multi-head and multi-query attention.

Reference: Ainslie et al., "GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints," arXiv:2305.13245.

Multi-Head Latent Attention (MLA)

- MQA and GQA shrink the cache by sharing key and value heads; aggressive sharing costs quality.
- MLA (DeepSeek-V2, 2024) instead compresses keys and values into one low-rank latent per token, caches only the latent, and reconstructs per-head keys and values at attention time.
- Expressivity stays close to full MHA with a far smaller cache: DeepSeek-V2 reports 93% less KV memory than its MHA baseline; used across the DeepSeek family (V2, V3, R1).

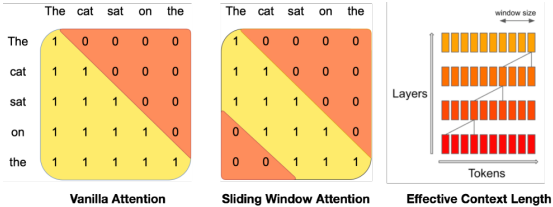


DeepSeek-AI, "DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model," arXiv:2405.04434,

Figure 3.

Sliding-Window and Hybrid Attention

- Sliding-window attention (Mistral 7B, 2023): each token attends only to the previous W positions ($W = 4096$ in Mistral), capping the per-layer KV cache at W entries no matter how long the context grows.
- Information still travels further than W: after L layers the receptive field reaches $L \times W$ tokens.
- The 2026 pattern is hybrid: stacks alternate windowed layers with occasional full-attention layers (Gemma, GPT-OSS), keeping long-range retrieval while most layers stay cheap.



Jiang et al., "Mistral 7B," arXiv:2310.06825, Figure 1.

Decoder-only LLMs (post-2017 trend)

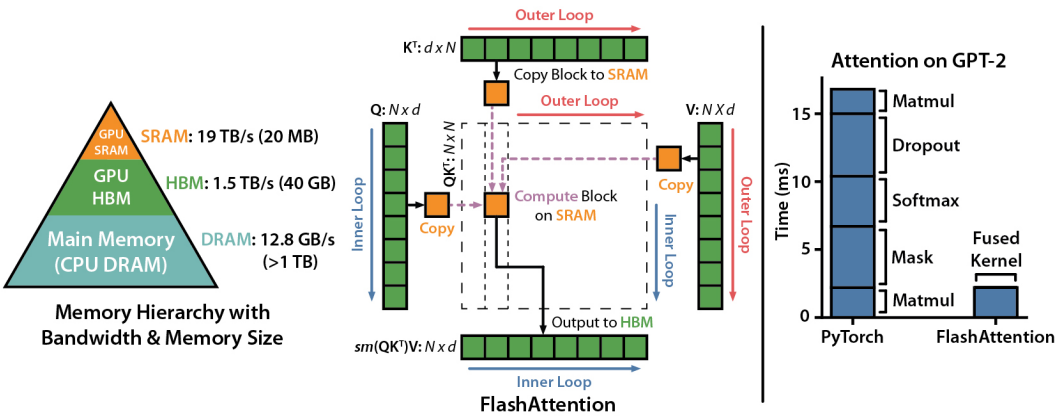
- GPT-style **causal (masked) self-attention**: each token attends only to the past.
- Single stack replaces separate encoder/decoder for many language (and vision–language) workloads.
- Encoder–decoder Transformers remain standard in **seq2seq** (MT, Whisper ASR, some VL captioning) where cross-attention helps.

FlashAttention

FlashAttention: The Core Idea

- Standard attention writes the full $N \times N$ score matrix to GPU main memory (HBM), reads it back for the softmax, then reads it again to multiply with V: most of the time goes to memory traffic, not math.
- On-chip SRAM is roughly an order of magnitude faster than HBM but only megabytes in size.
- FlashAttention (Dao et al., 2022) splits Q, K, V into tiles that fit in SRAM and computes attention block by block with a running softmax, never materializing the $N \times N$ matrix.
- The output is exact (no approximation), memory grows linearly instead of quadratically with sequence length, and wall-clock time drops because IO shrinks.

FlashAttention: Memory Hierarchy and Tiling



GPU memory hierarchy, tiling scheme, and GPT-2 kernel timings. Dao et al., "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness," arXiv:2205.14135; figure from the official flash-attention repository.

FlashAttention-2: Overview

- **Paper:** Dao et al. (2023)
- **Goal:** Memory-efficient and fast attention computation
- **Highlights:**
  - Up to 73% of theoretical peak FLOPs (72% model FLOPs utilization on A100)
  - 2–4$\times$ faster than standard attention
- **Techniques:**
  - Multi-query and grouped-query attention
  - Hardware-aware scheduling
  - Supports long context windows (32k+ tokens)
- **Adoption:**
  - Used in OpenAI GPT-4, Mistral, LLaMA 3
  - Critical for scaling to trillion-parameter models efficiently

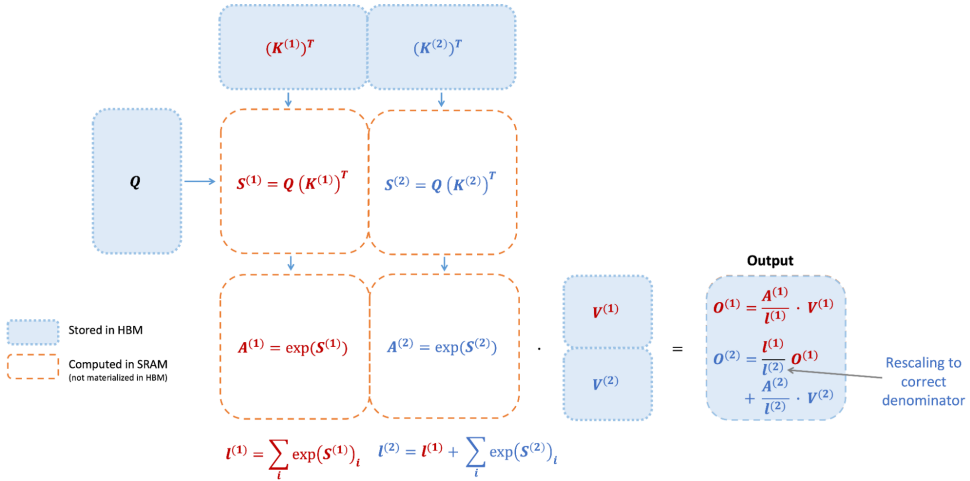
FlashAttention-2: Key Improvements

- **Fewer non-matmul FLOPs:**
  - Algorithm tweaks reduce non-matmul operations (e.g., rescaling, masking).
  - Maximizes use of specialized GPU units (Tensor Cores).
  - Non-matmul FLOPs are up to 16× more expensive than matmul FLOPs.
- **Better Parallelism:**
  - Parallelizes over sequence length in addition to batch size and heads.
  - Improves GPU utilization for long sequences and small batches.

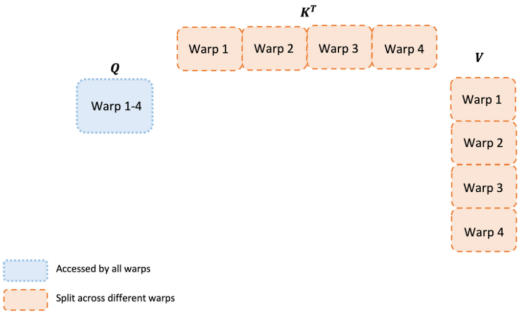
FlashAttention-2: Key Improvements (cont.)

- **Improved Work Partitioning:**
  - Splits Q across warps (instead of K/V), reducing shared memory communication.
  - Yields significant speedup by minimizing synchronization.
- **New Features:**
  - Supports head dimensions up to 256.
  - Enables multi-query and grouped-query attention.

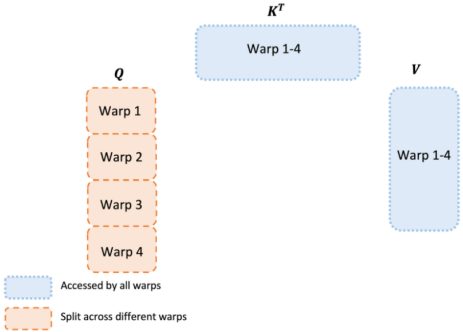
FlashAttention-2: Architecture



FlashAttention-2: Architecture (cont.)



(a) FlashAttention

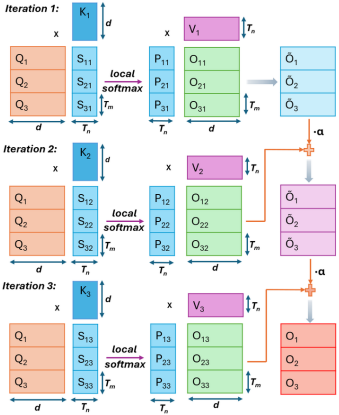


(b) FlashAttention-2

FlashAttention-2: Architecture (cont.)

- **MQA and GQA Support:** Works with the shared-KV attention variants from the previous section, combining kernel speed with a smaller cache.
- **Hardware-Aware Scheduling:** Optimizes GPU scheduling to maximize throughput and minimize latency.
- **Long Context Support:** Efficiently handles long sequences (32k+ tokens) with minimal memory overhead.

FlashAttention-2: Architecture (cont.)



Iterative update of output in FlashAttention-2 with Cn = 3 iterations.

FlashAttention-2: Performance Benchmarks

- **Speed:**
  - Up to 2× faster than FlashAttention-1, 9× faster than standard PyTorch attention.
  - Achieves up to 335 TFLOPs/s on H100 GPUs.
- **End-to-End Training:**
  - 1.3× speedup over optimized FlashAttention models.
  - Up to 225 TFLOPs/s on A100 GPU (72% model FLOPs utilization).
- **Example Results:**

Model	Baseline	FlashAttention	FlashAttention-2
GPT3-1.3B 2k	142	189	196
GPT3-1.3B 8k	72	170	220
GPT3-2.7B 2k	149	189	205
GPT3-2.7B 8k	80	175	225

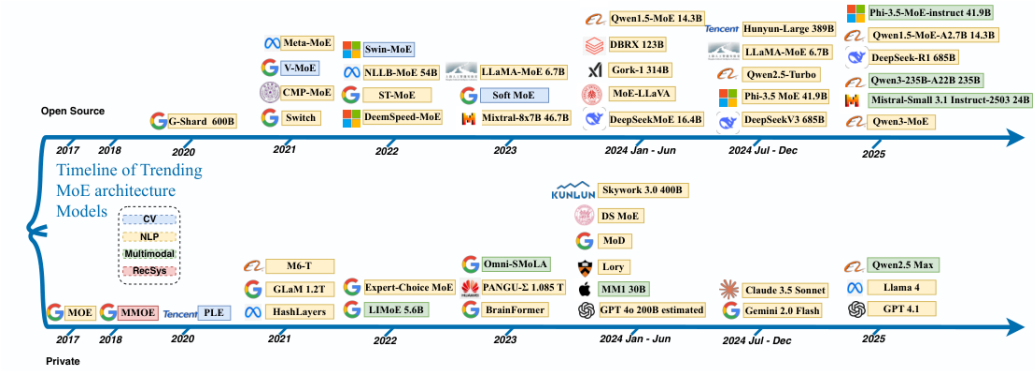
- Baseline: Megatron-LM without FlashAttention.

Mixture of Experts (MoE)

- A dense feed-forward block always runs the same large MLP for every token. An MoE block replaces that MLP with several smaller expert MLPs.
- A lightweight router (gating network) looks at each token and picks one or a few experts to run. The rest stay idle for that token.
- Why it helps: you can increase total parameters (model capacity) without increasing active FLOPs per token as much as a fully dense model—similar spirit to "widen capacity, keep per-token work bounded."
- Routing can become unbalanced, communication-heavy in distributed training, or collapse onto a few experts unless training is regularized.

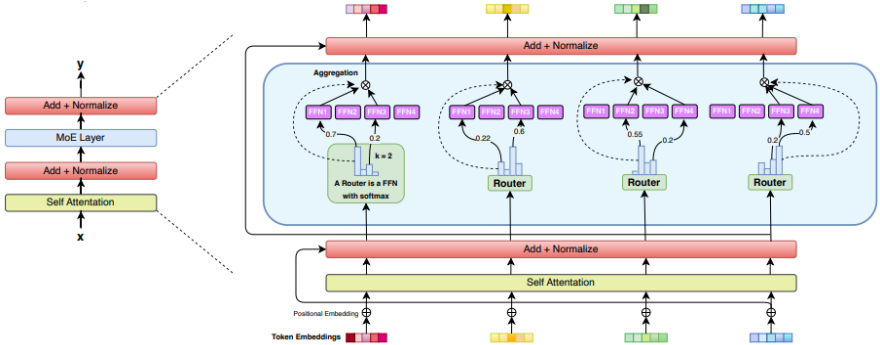
Sources: Zhang et al., "Mixture of Experts in Large Language Models," arXiv:2507.11181; Kranen & Nguyen, NVIDIA Technical Blog (Mar. 2024).

MoE milestones



Reference: Zhang et al., "Mixture of Experts in Large Language Models," arXiv:2507.11181.

MoE for a decoder-only Transformer

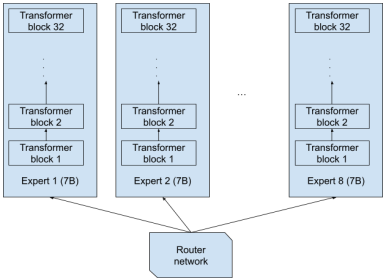


MoE for a decoder-only Transformer with $k = 2$ experts.

- Input **x** enters a **gating / router** that outputs scores for each expert.
- Only the top-k experts run their FFN; outputs are **combined** (often a weighted sum) for the residual stream.

Example: Mixtral 8×7B — a common misconception

- The name sounds like "eight separate 7B models." The **released architecture is not that**: experts are **not** eight full standalone stacks.
- Instead, think "**eight expert FFNs** per MoE layer, with **shared** attention and norms," and only a subset fires per token (here, **top-2** experts).



Reference: Kranen & Nguyen, "Applying Mixture of Experts in LLM Architectures," NVIDIA Technical Blog (2024), article.

MoE: Most Preferred Tokens By Experts

Reference: Kranen & Nguyen, "Applying Mixture of Experts in LLM Architectures," NVIDIA Technical Blog (2024), article.

Vision–Language Models: why pair vision with language?

- Strong **image embeddings** are useful, but many tasks still need a way to **say what you want** in words (“what species of bird is this?”, “read the sign in the corner”).
- **Language** makes it easy to specify new tasks without collecting a fresh classification label set for every concept—a path to flexible, user-steerable vision AI.
- **Nomenclature (informal):** an **LLM** is language-first; a **VLM** adds a **vision encoder** so the stack can **condition on images**; an **MLLM** usually means several modalities (image, video, audio) in one family.

Bandaru, *Vision Language Models* (blog, Aug. 2025), rohitbandaru.github.io/blog/Vision-Language-Models/.

Fusion Patterns: Where do image tokens meet the LLM?

- Early fusion: build one long token sequence (visual tokens + text tokens) and run attention over the mixture—what most "native" VLMs aim for today.
- Middle fusion: insert vision cross-attention inside some transformer blocks (common when adapting a legacy text-only stack such as early LLaMA 3.2).
- Late fusion: combine modalities only at the end—usually too weak for open-ended captioning/VQA because the LLM never gets rich visual state early enough.
- Images create many tokens; good designs compress visual length (pooling, grouping, tiling) before the LLM sees them.

Early Fusion approaches: MLP Adapter

- Freeze (or lightly train) a strong ViT; map its patch tokens through a small MLP projector into the LLM embedding width, then concatenate with text tokens.
- Simple, stable, and very common in open recipes (LLaVA, LLaMA 4-style early fusion, PaliGemma 2, DeepSeek-VL, . . .).

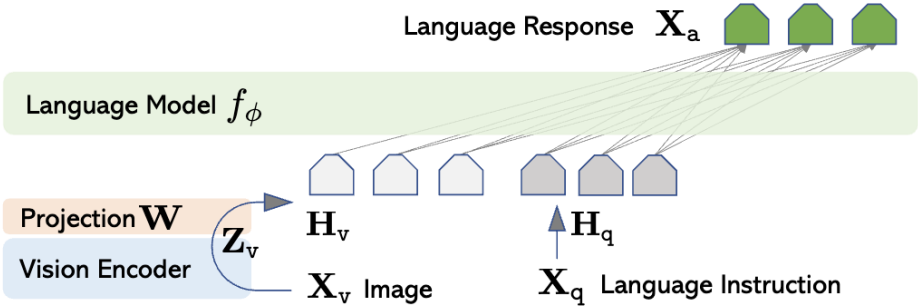


Diagram from Bandaru. Paper: Liu et al., LLaVA, arXiv:2304.08485.

Early Fusion approaches: Cross-Attention

- A small **adapter** uses learned query vectors that **cross-attend** to ViT tokens and output a **fixed small** set of visual summaries for the LLM.

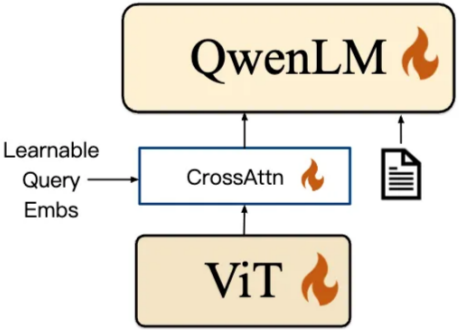


Diagram from Bandaru (Qwen-VL "position-aware" adapter).

Middle Fusion Approaches: Flamingo

- Keep a pretrained ViT and LLM frozen; insert Perceiver resamplers (compress variable tokens → fixed count) and gated cross-attention blocks inside the LLM stack.

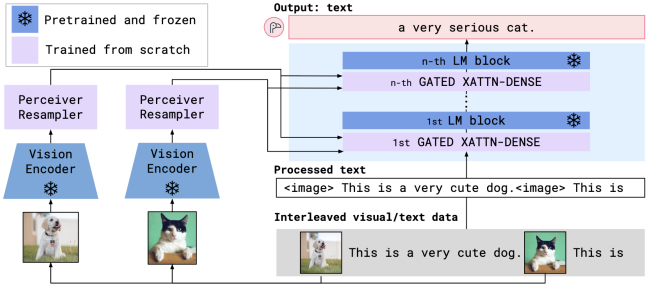


Diagram from Bandaru. Paper: Alayrac et al., Flamingo, arXiv:2204.14198.

Mid Fusion Approaches: LLaMA 3

- **ViT encoder:** 32-layer ViT trained CLIP-style for image-text alignment; features extracted from multiple layers.
- **Decoder:** Image cross attention in every fourth block (no gating on cross attention).
- **Mid fusion** suits LLaMA 3's text-first design; LLaMA 4 moves to early fusion (MLP adapter) for true multimodal training.

Reasoning Models and Thinking Tokens (Recap)

- A 2017 Transformer spends one forward pass per generated token: every query receives the same compute, easy or hard.
- Reasoning models (OpenAI o1, DeepSeek-R1) generate a long chain of thought before the final answer, so hard questions receive more decode steps than easy ones.
- Accuracy scales with test-time compute: thinking longer or sampling more candidate solutions buys accuracy without retraining.
- R1 shows this allocation can be learned: reinforcement learning on verifiable rewards teaches the model when and how long to think.

DeepSeek-AI, "DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning," arXiv:2501.12948.

What Thinking Tokens Mean for the Architecture

- **Cost model:** serving cost is now dominated by decode length, not just parameter count. A smaller model that thinks longer can beat a larger one that answers immediately.
- **KV cache:** every thinking token adds cache entries, so long traces multiply the memory pressure that GQA, MLA, and sliding-window attention exist to relieve.
- **Kernels:** attention runs over reasoning traces tens of thousands of tokens long, exactly the regime FlashAttention targets.
- **Sparsity:** MoE keeps per-token FLOPs bounded, so capacity can grow without making each thinking token slower.
- The 2026 design trade: spend compute at training time (a bigger model) or at inference time (longer thinking), tuned per deployment.

Summary: Transformers 2017 vs. 2026

Component	2017 Transformer	2026 consensus
Attention mechanism	Multi-Head Attention (MHA)	Grouped-Query Attention (GQA) / Multi-Head Latent Attention (MLA)
Position encoding	Absolute sinusoidal (or learned absolute)	Rotary Positional Embeddings (RoPE)
Normalization	Post-Layer Normalization (post-LN)	Pre-Layer Normalization with RMSNorm
Activation	ReLU in the FFN	SwiGLU / gated linear units
Bias terms	Biases in linear layers common	Bias-free or minimal-bias architectures
Parameter density	Dense (all parameters active per forward)	Sparse activation (e.g., Mixture-of-Experts)
Attention span	Every token attends to all positions	Hybrid stacks: sliding-window layers + occasional full-attention layers
Inference compute	Fixed: one forward pass per token	Test-time scaling: thinking tokens on hard queries

Summary

- The slowest part is often getting data in and out of memory, not just the math.
- New models (2024–2026) are designed together, optimizing attention, normalization, position encoding, sparsity (MoE), and serving (memory, paging, kernels).
- The main goals: cut down on moving data, adjust compute to the input's length and type, and add more parameters without needing a lot more computation per token.

References and Further Reading

1. Vaswani et al., “Attention Is All You Need”, NeurIPS 2017, arXiv:1706.03762.
2. Su et al., “RoFormer: Enhanced Transformer with Rotary Position Embedding”, arXiv:2104.09864.
3. Touvron et al., “LLaMA: Open and Efficient Foundation Language Models”, arXiv:2302.13971; “LLaMA 2: Open Foundation and Fine-Tuned Chat Models”, arXiv:2307.09288.
4. Shazeer, “Fast Transformer Decoding: One Write-Head is All You Need”, arXiv:1911.02150 (MQA).
5. Ainslie et al., “GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints”, arXiv:2305.13245.
6. Fedus et al., “Switch Transformers: Scaling to Trillion Parameter Models”, JMLR 2022, arXiv:2101.03961.
7. Dao et al., “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness”, NeurIPS 2022, arXiv:2205.14135; FlashAttention-2, arXiv:2307.08691.
8. Liu et al., “Visual Instruction Tuning” (LLaVA), arXiv:2304.08485.
9. Kwon et al., “Efficient Memory Management for Large Language Model Serving with PagedAttention”, arXiv:2309.06180 (vLLM).
10. Zhang et al., “Mixture of Experts in Large Language Models”, arXiv:2507.11181.
11. Shazeer, “GLU Variants Improve Transformer”, arXiv:2002.05202 (SwiGLU).
12. Zhang & Sennrich, “Root Mean Square Layer Normalization”, arXiv:1910.07467.

References and Further Reading (cont.)

13. Press et al., "Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation", arXiv:2108.12409 (ALiBi).
14. DeepSeek-AI, "DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model", arXiv:2405.04434 (MLA).
15. Jiang et al., "Mistral 7B", arXiv:2310.06825 (sliding-window attention).
16. DeepSeek-AI, "DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning", arXiv:2501.12948.
17. Kranen & Nguyen, "Applying Mixture of Experts in LLM Architectures," NVIDIA Technical Blog, 2024, developer.nvidia.com/blog/....
18. Bandaru, "Vision Language Models," blog, Aug. 2025, rohitbandaru.github.io/blog/Vision-Language-Models/.
19. Sorte (Medium), Transformer architectures in 2026: foundations and resources.
20. LLMOrbit (circular taxonomy of LLM systems).
21. Aman (Medium), Inside frontier open-weight LLM architectures.
22. MDPI survey, Mapping the LLM landscape (architectures and benchmarks).
23. MIT course notes, Transformers (6.390).

References and Further Reading (cont.)

24. Shui, MHA vs. MQA vs. GQA.
25. DataHacker.rs, Modern transformer architectures (LLMs from scratch series).
26. Hugging Face blog, What changed in the Transformer architecture.
27. Tan, The crystallization of transformer architectures (2017–2025).
28. Bandaru, Transformer design guide (modern architecture).
29. NVIDIA, Mastering LLM techniques: inference optimization.
30. Gencc (Medium), KV cache, PagedAttention, and MLA.
31. PythonAlchemist, Attention variants explained (MHA, GQA, MQA, MLA, SWA, ...).
32. Tri Dao, FlashAttention-3 (PDF); NeurIPS 2024 proceedings entry for FA3.
33. Lightly, Introduction to vision–language models.
34. Stanford CS224N, NLP with deep learning (course hub).
35. Marek Suppa, NLP 2026 (teaching resources).